Walking each waypoint scheme through a 112-step episode

Imagine one TwoRoom-style trajectory with **112 raw env steps**, numbered 1 to 112:

```
1 2 3 4 5 6 7 8 9 10 ... 50 ... 100 ... 112
+------------ 112 raw env steps -------------+
```

A "waypoint" is just a step number we pick to be a special anchor. Between two consecutive waypoints, the actions get packed into one **macro-action**. Each scheme picks waypoints differently.

Scheme 1 — PLDM (the closest analog to LeWM, Appendix B.4)

Rule: chop out exactly 60 raw steps, then place waypoints every 10 steps. Deterministic.

```
Step 1: Pick a starting point, say step 1.
Step 2: Cut out a window of 60 steps:
        steps  1 -- 60                      (61..112 unused for this sample)
Step 3: Place waypoints at fixed stride 10:
        positions: 1, 11, 21, 31, 41, 51    <- 6 waypoints
Step 4: Segments between waypoints:
        [10, 10, 10, 10, 10]                <- all equal, every time
```

Visual:

```
 1    11   21   31   41   51       60                                  112
 W----W----W----W----W----W        |
 | 10 | 10 | 10 | 10 | 10 |
 +--- always identical segments ---+
```

Every training sample looks like this. **Zero within-sample variance.** Across samples, the only thing that changes is the starting point (1, 2, 3, ...) — segments are always 10 steps long.

Scheme 2 — DINO-WM (Appendix B.3)

Rule: randomize the *total span* per sample, then place 5 waypoints inside it.

```
Step 1: Sample segment length L ~ Uniform(25, 70). Say L = 50.
Step 2: Pick a starting point. Say step 1. Window is steps 1..50.
Step 3: Place 5 waypoints inside the 50-step window
        (paper doesn't say how -- likely roughly uniform).
        Example: positions 1, 13, 25, 37, 50.
Step 4: Segments:
        [12, 12, 12, 13]                    <- roughly equal, 1-step jitter
```

Visual (one possible draw):

```
 1        13        25        37        50                                 112
 W---------W---------W---------W---------W
 |   12    |   12    |   12    |   13    |
```

Across samples: span shifts (some L=27, some L=68), but within a single sample, the four segments stay similar. **Bounded within-sample variance.** The total span never grows past 70 or shrinks below 25.

Scheme 3 — VJEPA2-AC (Appendix B.2)

Rule: sample a duration in seconds (0.33s to 4s), use only **3 waypoints** with the **middle one chosen uniformly at random**.

```
Step 1: Sample segment length. Say 30 steps.
Step 2: Pick a starting point. Say step 1. Window is steps 1..30.
Step 3: First and last waypoints are the endpoints.
        Middle waypoint = uniform random somewhere inside.
        Example: middle at step 12.
        Positions: 1, 12, 30.
Step 4: Segments: [11, 18]                  <- 2 segments only
```

Visual:

```
1            12                  30                                  112
W------------W-------------------W
|     11     |        18         |
```

Only 2 segments per sample. Their ratio is whatever uniform-random luck draws — could be near-equal [14, 15] or lopsided [3, 26]. Bounded but noisy.

Scheme 4 — Our current code

Rule: fixed 100-step window after frameskip, but waypoints inside are placed by `torch.randperm` with no minimum gap.

```
Step 1: Cut out exactly 100 raw steps (span = frameskip*num_steps = 5*20).
        Say starting at step 1. Window: 1..100.
Step 2: The dataloader keeps every 5th step (frameskip). So our 'frames' are at
        raw steps 1, 6, 11, 16, ..., 96. That's 20 frames, indexed 0..19.
Step 3: sample_waypoints(T=20, N=3) returns N+2 = 5 frame indices.
        Always includes frame 0 and frame 19. Picks 3 more uniformly at random.
```

Now the question is **which 3 interior frames get picked.** Here are two possible draws:

Draw A (typical)

```
frame indices: [0, 4, 11, 16, 19]
raw steps:     [1, 21, 56, 81, 96]
segments (raw steps): [20, 35, 25, 15]
```

```
 1          21                       56                  81           96    100        112
 W----------W------------------------W-------------------W------------W
 |     20    |           35           |         25         |      15     |
```

Segment ratio 35:15 = **2.3x**. A bit uneven but tolerable.

Draw B (pathological — also legal)

```
frame indices: [0, 1, 2, 3, 19]
raw steps:     [1, 6, 11, 16, 96]
segments (raw steps): [5, 5, 5, 80]
```

```
 1 6 11 16                                                  96      100       112
 WWWWW-------------------------------------------------------W
 |5|5|5|                           80                        |
```

Segment ratio 80:5 = **16x**. A_psi is asked to compress *one effective action* (5 raw steps) into a macro-action for the first three segments, then compress *sixteen effective actions* (80 raw steps) into a macro-action for the last one — all in the same training example.

Side-by-side on the 112-step episode

Scheme	Window	# waypoints	# segments	Example segments	Max ratio in one sample
PLDM	fixed 60	6	5	[10, 10, 10, 10, 10]	1x (always equal)
DINO-WM	Unif(25, 70)	5	4	[12, 12, 12, 13]	~1.1x
VJEPA2-AC	Unif(3, 40)	3	2	[11, 18]	a few x
Our code	fixed 100	5	4	[20, 35, 25, 15] typical	typ ~2.5x, worst 16x

---

The point

PLDM — the JEPA-from-pixels backbone, which is the design closest to LeWM — picks waypoints **at fixed stride 10**, every time. No randomness inside the sample. Our code's "random interior frames, no minimum gap" gives the model wildly inconsistent training signals about what one macro-action represents (sometimes 5 raw steps, sometimes 80).

That's the concern.

Fix: replace `torch.randperm` with a fixed stride. For our T=20, N=3 setup, that means picking frame indices **[0, 5, 10, 15, 19]** (or [0, 4, 9, 14, 19]) every sample — same spec PLDM used in HWM.